

AegisLab

Automated API Security Test Report

Target: http://127.0.0.1:9000

Scan ID: a757042e-8689-4248-a11d-d33c4d9e6f7e

Generated: 2026-07-15 08:14:35

Security Score	Grade	Endpoints Tested	Total Findings
0/100	F	8	17

Findings by Severity

Severity	Count
CRITICAL	7
HIGH	7
MEDIUM	2
LOW	1

OWASP Category Breakdown

Category	Findings	Score Impact
API2:2023 Broken Authentication	7	-43.7
API1:2023 Broken Object Level Authorization	1	-10.0
A03:2021 Injection	1	-10.0
API4:2023 Unrestricted Resource Consumption	1	-10.0
API3:2023 Broken Object Property Level Authorization	3	-27.8
API8:2023 Security Misconfiguration	2	-21.2
A02:2021 Cryptographic Failures	1	-20.0
API7:2023 Server Side Request Forgery	1	-20.0

Detailed Findings

[CRITICAL] Protected endpoint accessible without authentication

Endpoint: GET /users/{id} **OWASP:** API2:2023 Broken Authentication **Module:** auth_tests

Description: The endpoint GET /users/{id} is expected to require authentication (per the imported spec) but returned HTTP 200 with no Authorization header supplied.

Impact: Unauthenticated users may read or modify data that should be restricted to logged-in users.

Recommendation: Enforce an authentication guard/middleware on this route and verify it runs before any business logic. Add an automated test that asserts 401 on missing credentials.

[CRITICAL] Protected endpoint accessible without authentication

Endpoint: POST /users **OWASP:** API2:2023 Broken Authentication **Module:** auth_tests

Description: The endpoint POST /users is expected to require authentication (per the imported spec) but returned HTTP 200 with no Authorization header supplied.

Impact: Unauthenticated users may read or modify data that should be restricted to logged-in users.

Recommendation: Enforce an authentication guard/middleware on this route and verify it runs before any business logic. Add an automated test that asserts 401 on missing credentials.

[CRITICAL] Protected endpoint accessible without authentication

Endpoint: POST /fetch-avatar **OWASP:** API2:2023 Broken Authentication **Module:** auth_tests

Description: The endpoint POST /fetch-avatar is expected to require authentication (per the imported spec) but returned HTTP 200 with no Authorization header supplied.

Impact: Unauthenticated users may read or modify data that should be restricted to logged-in users.

Recommendation: Enforce an authentication guard/middleware on this route and verify it runs before any business logic. Add an automated test that asserts 401 on missing credentials.

[CRITICAL] Dangerous CORS configuration: wildcard origin with credentials allowed

Endpoint: GET /users/{id} **OWASP:** API8:2023 Security Misconfiguration **Module:** data_exposure_tests

Description: The API returns Access-Control-Allow-Origin: * together with Access-Control-Allow-Credentials: true.

Impact: Most browsers will reject this combination, but where supported it lets ANY website read authenticated responses on behalf of a logged-in victim, leading to full account compromise via CSRF-style requests.

Recommendation: Echo back a specific allow-listed origin (never *) whenever Allow-Credentials is true.

[CRITICAL] API base URL uses unencrypted HTTP

Endpoint: N/A (base URL) **OWASP:** A02:2021 Cryptographic Failures **Module:** transport_security_tests

Description: The configured base URL (http://127.0.0.1:9000) uses http:// rather than https://.

Impact: All traffic — including bearer tokens and request/response bodies — travels in plaintext and can be intercepted or modified in transit.

Recommendation: Serve the API exclusively over HTTPS/TLS 1.2+ and redirect all HTTP traffic to HTTPS.

[CRITICAL] Mass assignment: unexpected field 'role' accepted and reflected

Endpoint: POST /users **OWASP:** API3:2023 Broken Object Property Level Authorization **Module:** mass_assignment_tests

Description: Sending an undocumented 'role' field (value: 'admin') alongside the normal request body was accepted (HTTP 200), and the response echoed it back — meaning the server stored or applied a field the client should never have been able to set directly.

Impact: A real attacker could set this field to escalate privileges (role/isAdmin), manipulate pricing or balances, or flip verification/premium flags — entirely through a normal-looking write request, without any other vulnerability needed.

Recommendation: Use an explicit allowlist (DTO / whitelist pattern) for writable fields instead of binding the raw request body to your model. Server-controlled fields (role, price, verified status, etc.) must never be settable from client input at all.

[CRITICAL] Server-side request forgery via 'avatar_url' — cloud metadata reachable

Endpoint: POST /fetch-avatar **OWASP:** API7:2023 Server Side Request Forgery **Module:** ssrf_tests

Description: Pointing the 'avatar_url' body parameter at the cloud instance metadata address (169.254.169.254) caused the response to include metadata-service content (ami-id) — the server fetched an attacker-supplied URL itself.

Impact: If this API runs on AWS/GCP/Azure, an attacker can use this to steal the instance's IAM credentials from the metadata service, potentially compromising far more than just this API (full cloud account access depending on the role's permissions).

Recommendation: Never fetch a raw user-supplied URL server-side. Use an allowlist of permitted hosts/schemes, resolve and reject requests to link-local/private IP ranges (169.254.0.0/16, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8) before connecting, and disable HTTP redirects when following user-supplied URLs.

[HIGH] Endpoint accepts a malformed/invalid bearer token

Endpoint: GET /users/{id} **OWASP:** API2:2023 Broken Authentication **Module:** auth_tests

Description: Sending a syntactically invalid bearer token to /users/{id} returned HTTP 200 instead of a 401 Unauthorized.

Impact: Indicates token validation may be missing, fail-open, or only checked on some code paths.

Recommendation: Verify JWT/opaque token validation runs on every request and fails closed on any parse/verify error.

[HIGH] Endpoint accepts a malformed/invalid bearer token

Endpoint: POST /users **OWASP:** API2:2023 Broken Authentication **Module:** auth_tests

Description: Sending a syntactically invalid bearer token to /users returned HTTP 200 instead of a 401 Unauthorized.

Impact: Indicates token validation may be missing, fail-open, or only checked on some code paths.

Recommendation: Verify JWT/opaque token validation runs on every request and fails closed on any parse/verify error.

[HIGH] Endpoint accepts a malformed/invalid bearer token

Endpoint: POST /fetch-avatar **OWASP:** API2:2023 Broken Authentication **Module:** auth_tests

Description: Sending a syntactically invalid bearer token to /fetch-avatar returned HTTP 200 instead of a 401 Unauthorized.

Impact: Indicates token validation may be missing, fail-open, or only checked on some code paths.

Recommendation: Verify JWT/opaque token validation runs on every request and fails closed on any parse/verify error.

[HIGH] Possible missing object-ownership check (IDOR/BOLA)

Endpoint: GET /users/{id} **OWASP:** API1:2023 Broken Object Level Authorization **Module:** authorization_tests

Description: Requesting /users/{id} with different ID values (1, 2) using the SAME token returned HTTP 200 with distinct payloads for each, with no apparent ownership/ACL check rejecting access to records the caller may not own.

Impact: A user may be able to read other users' records simply by incrementing an ID, exposing private data.

Recommendation: Add an explicit ownership/ACL check in the handler (e.g. WHERE owner_id = :current_user) rather than relying on the ID being 'hard to guess'. Return 403/404 for records the caller doesn't own.

[HIGH] Possible NoSQL operator injection via body field 'email'

Endpoint: POST /users **OWASP:** A03:2021 Injection **Module:** injection_tests

Description: Replacing the value of 'email' with a MongoDB query operator ({'\$ne': None}) was accepted (HTTP 200) instead of being rejected as an invalid string value.

Impact: If the field reaches a NoSQL query unsanitized, an attacker could bypass auth checks or filters (e.g. login bypass via \$ne).

Recommendation: Validate and coerce input types strictly (reject objects where a string/number is expected) before passing to the database driver.

[HIGH] No rate limiting detected on sensitive endpoint

Endpoint: POST /auth/login **OWASP:** API4:2023 Unrestricted Resource Consumption **Module:** rate_limit_tests

Description: 20 consecutive requests to POST /auth/login all succeeded without ever receiving an HTTP 429 (Too Many Requests) response.

Impact: Without rate limiting, this endpoint is exposed to credential stuffing, brute-force, OTP-guessing, or resource-exhaustion attacks.

Recommendation: Add per-IP and per-account rate limiting (e.g. token-bucket or sliding-window) on this route, returning 429 with a Retry-After header once the limit is exceeded.

[HIGH] Response body includes sensitive/credential-like fields

Endpoint: GET /users/{id} **OWASP:** API3:2023 Broken Object Property Level Authorization **Module:** data_exposure_tests

Description: The response from `/users/{id}` includes one or more fields that look like secrets or credentials: `password_hash`.

Impact: Sensitive fields such as password hashes or tokens should never be serialized to API clients; exposure can lead to account takeover if leaked further (logs, browser devtools, caches).

Recommendation: Use an explicit response DTO/serializer that allow-lists exposed fields, rather than returning the raw database model.

[MEDIUM] Login error messages leak account existence

Endpoint: `POST /auth/login` **OWASP:** API2:2023 Broken Authentication **Module:** `auth_tests`

Description: The login endpoint returns distinguishable responses for 'unknown account' vs 'wrong password', allowing account enumeration.

Impact: Attackers can build a list of valid/registered email addresses for targeted phishing or credential stuffing.

Recommendation: Return an identical generic message (e.g. 'Invalid email or password') and identical status code/timing for both cases.

[MEDIUM] Response appears to expose internal/implementation fields (excessive data exposure)

Endpoint: `GET /users/{id}` **OWASP:** API3:2023 Broken Object Property Level Authorization **Module:** `data_exposure_tests`

Description: The response includes internal-looking fields not typically meant for API consumers: `__v`, `password_hash`.

Impact: Returning raw database rows leaks schema/implementation details and may include fields with no intended external use, increasing attack surface for object-property-level attacks.

Recommendation: Map database models to dedicated response DTOs and explicitly select only the fields the API contract requires.

[LOW] X-Powered-By header reveals backend framework

Endpoint: `GET /users/{id}` **OWASP:** API8:2023 Security Misconfiguration **Module:** `data_exposure_tests`

Description: The response includes an X-Powered-By header value of 'Express/4.17.1'.

Impact: Knowing the exact framework/version helps attackers select known exploits faster.

Recommendation: Disable the X-Powered-By header (e.g. `app.disable('x-powered-by')` in Express, or equivalent middleware config).
